

**Московский государственный технический
университет им. Н.Э. Баумана**

Факультет информатика и системы управления
Кафедра системы обработки информации и управления

Курс «Парадигмы и конструкции языков программирования»
Отчет по лабораторной работы №2

Выполнил:
студент группы ИУ5-32Б:
Кунев В.
Подпись и дата:

Проверил:
преподаватель каф. ИУ5
Нардид А.Н.
Подпись и дата:

Москва, 2025 г.

Описание задания

Цель лабораторной работы: изучение объектно-ориентированных возможностей языка Python.

Задание:

1. Необходимо создать виртуальное окружение и установить в него хотя бы один внешний пакет с использованием `pip`.
2. Необходимо разработать программу, реализующую работу с классами. Программа должна быть разработана в виде консольного приложения на языке Python 3.
3. Все файлы проекта (кроме основного файла `main.py`) должны располагаться в пакете `lab_python_oop`.
4. Каждый из нижеперечисленных классов должен располагаться в отдельном файле пакета `lab_python_oop`.
5. Абстрактный класс «Геометрическая фигура» содержит абстрактный метод для вычисления площади фигуры. Подробнее про абстрактные классы и методы Вы можете прочитать [здесь](#).
6. Класс «Цвет фигуры» содержит свойство для описания цвета геометрической фигуры. Подробнее про описание свойств Вы можете прочитать [здесь](#).
7. Класс «Прямоугольник» наследуется от класса «Геометрическая фигура». Класс должен содержать конструктор по параметрам «ширина», «высота» и «цвет». В конструкторе создается объект класса «Цвет фигуры» для хранения цвета. Класс должен переопределять метод, вычисляющий площадь фигуры.
8. Класс «Круг» создается аналогично классу «Прямоугольник», задается параметр «радиус». Для вычисления площади используется константа `math.pi` из модуля [math](#).
9. Класс «Квадрат» наследуется от класса «Прямоугольник». Класс должен содержать конструктор по длине стороны. Для классов «Прямоугольник», «Квадрат», «Круг»:
 - Определите метод `__repr__`, который возвращает в виде строки основные параметры фигуры, ее цвет и площадь. Используйте метод `format` - <https://pyformat.info/>
 - Название фигуры («Прямоугольник», «Квадрат», «Круг») должно задаваться в виде поля данных класса и возвращаться методом класса.
10. В корневом каталоге проекта создайте файл `main.py` для тестирования Ваших классов (используйте следующую конструкцию - <https://docs.python.org/3/library/main.html>). Создайте следующие объекты и

выведите о них информацию в консоль (N - номер Вашего варианта по списку группы):

- Прямоугольник синего цвета шириной N и высотой N.
- Круг зеленого цвета радиусом N.
- Квадрат красного цвета со стороной N.
- Также вызовите один из методов внешнего пакета, установленного с использованием `pip`.

11. Дополнительное задание. Протестируйте корректность работы Вашей программы с помощью модульного теста.

Текст программы

`main.py`:

```
#!/usr/bin/env python3
"""
Основной файл для тестирования классов геометрических фигур
"""
import sys
import os

# Импортируем внешний пакет
from colorama import Fore, Style, init

from lab_python_oop.rectangle import Rectangle
from lab_python_oop.circle import Circle
from lab_python_oop.square import Square

# Добавляем путь к пакету
sys.path.insert(0, os.path.join(os.path.dirname(__file__)))

def main():
    """
    Основная функция тестирования
    """
    # Инициализируем colorama
    init(autoreset=True)

    # Номер варианта
    n = 13

    print(Fore.CYAN + "=== Лабораторная работа №2 ===")
    print(Fore.CYAN + "=== Объектно-ориентированные возможности Python ===")
    print()

    # Создаем объекты фигур
    try:
        rectangle = Rectangle(n, n, "синий")
        circle = Circle(n, "зеленый")
        square = Square(n, "красный")

        # Выводим информацию о фигурах
        print(Fore.YELLOW + "Информация о геометрических фигурах:")
        print(Fore.GREEN + f"1. {rectangle}")
        print(Fore.GREEN + f"2. {circle}")
        print(Fore.GREEN + f"3. {square}")
        print()

        # Демонстрация работы внешнего пакета colorama
        print(Fore.MAGENTA + "=== Демонстрация внешнего пакета colorama ===")
```

```

print(Fore.RED + "Этот текст красного цвета")
print(Fore.BLUE + "Этот текст синего цвета")
print(Fore.GREEN + "Этот текст зеленого цвета")
print(Style.BRIGHT + Fore.WHITE + "Этот текст ярко-белый")
print()

# Дополнительная информация о фигурах
print(Fore.CYAN + "=== Дополнительная информация ===")
print(f"Тип прямоугольника: {type(rectangle).__name__}")
print(f"Тип круга: {type(circle).__name__}")
print(f"Тип квадрата: {type(square).__name__}")
print()

# Проверка наследования
print(Fore.CYAN + "=== Проверка наследования ===")
print(f"Квадрат является прямоугольником: {isinstance(square,
Rectangle})}")
print(f"Площадь квадрата: {square.area():.2f}")

# Да, я жулик, но в данном коде сложно предположить возможные исключения
except Exception as e: # pylint: disable=broad-except
    print(Fore.RED + f"Ошибка: {e}")
    return 1

return 0

if __name__ == "__main__":
    sys.exit(main())

```

`test\_figures.py`:

```

"""
Тесты для геометрических фигур
"""

import unittest
import sys
import os

from lab_python_oop.rectangle import Rectangle
from lab_python_oop.circle import Circle
from lab_python_oop.square import Square

# Добавляем путь к пакету
sys.path.insert(0, os.path.join(os.path.dirname(__file__)))

class TestFigures(unittest.TestCase):

```

```

"""Тесты для геометрических фигур"""

def test_rectangle_area(self):
    """Тест площади прямоугольника"""
    rect = Rectangle(5, 10, "синий")
    self.assertEqual(rect.area(), 50)

def test_circle_area(self):
    """Тест площади круга"""
    circle = Circle(5, "зеленый")
    self.assertAlmostEqual(circle.area(), 78.539816, places=5)

def test_square_area(self):
    """Тест площади квадрата"""
    square = Square(5, "красный")
    self.assertEqual(square.area(), 25)

def test_square_inheritance(self):
    """Тест наследования квадрата от прямоугольника"""
    square = Square(5, "красный")
    self.assertIsInstance(square, Rectangle)

def test_color_property(self):
    """Тест свойства цвета"""
    rect = Rectangle(5, 10, "синий")
    self.assertEqual(rect.color.color, "синий")

    # Тест изменения цвета
    rect.color.color = "красный"
    self.assertEqual(rect.color.color, "красный")

def test_figure_names(self):
    """Тест названий фигур"""
    rect = Rectangle(5, 10, "синий")
    circle = Circle(5, "зеленый")
    square = Square(5, "красный")

    self.assertEqual(rect.name, "Прямоугольник")
    self.assertEqual(circle.name, "Круг")
    self.assertEqual(square.name, "Квадрат")

if __name__ == "__main__":
    unittest.main()

```

`lab\_python\_oop/\_\_init\_\_.py`:

"""

Пакет для лабораторной работы №2

Объектно-ориентированные возможности Python

"""

```
from .figure import Figure
from .color import Color
from .rectangle import Rectangle
from .circle import Circle
from .square import Square
```

```
__all__ = ["Figure", "Color", "Rectangle", "Circle", "Square"]
__version__ = "1.0.0"
```

`lab\_python\_oop/circle.py`

"""Модуль с классом Круг"""

```
import math
from typing import Union

from .figure import Figure
from .color import Color
```

```
class Circle(Figure):
    """Класс Круг"""
```

```
    FIGURE_NAME: str = "Круг"
```

```
    def __init__(self, radius: Union[int, float], color: str):
        self.radius = radius
        self.color: Color = Color(color)
```

```
    def area(self) -> float:
        """Вычисление площади круга"""
        return math.pi * self.radius**2
```

```
    @property
    def name(self) -> str:
        """Название фигуры"""
        return self.FIGURE_NAME
```

```
    def __repr__(self):
        # f-строки быстрее, но по заданию нужно использовать format
        return "{} {} цвета радиусом {}. Площадь: {:.2f}".format( # pylint:
            disable=consider-using-f-string
            self.name, self.color.color, self.radius, self.area()
        )
```

```
`lab_python_oop/color.py`
```

```
"""Модуль с классом Цвет фигуры"""
```

```
class Color:
    """Класс Цвет фигуры"""

    def __init__(self, color: str):
        self._color: str = color

    @property
    def color(self) -> str:
        """Свойство для получения цвета"""
        return self._color

    @color.setter
    def color(self, value: str):
        """
        Свойство для установки цвета

        Args:
            value (str): Значение цвета
        """
        self._color = value
```

```
`lab_python_oop/figure.py`
```

```
"""Модуль с абстрактным классом Геометрическая фигура"""
```

```
from abc import ABC, abstractmethod
```

```
class Figure(ABC):
    """Абстрактный класс Геометрическая фигура"""

    @abstractmethod
    def area(self):
        """Абстрактный метод для вычисления площади фигуры"""
        pass

    @property
    @abstractmethod
    def name(self):
        """Абстрактное свойство для получения названия фигуры"""
        pass

    def __repr__(self):
        # f-строки быстрее, но по заданию нужно использовать format
```



```

        return "{} {} цвета площадью {:.2f}".format( # pylint: disable=consider-
using-f-string
            self.name, self.color.color, self.area()
        )

```

```

`lab_python_oop/recta`"""Модуль с классом Прямоугольник"""

```

```

from typing import Union

```

```

from .figure import Figure
from .color import Color

```

```

class Rectangle(Figure):

```

```

    """Класс Прямоугольник"""

```

```

    FIGURE_NAME: str = "Прямоугольник"

```

```

    def __init__(self, width: Union[int, float], height: Union[int, float],
color: str):

```

```

        self.width = width
        self.height = height
        self.color: Color = Color(color)

```

```

    def area(self) -> Union[int, float]:
        """Вычисление площади прямоугольника"""
        return self.width * self.height

```

```

    @property

```

```

    def name(self) -> str:
        """Название фигуры"""
        return self.FIGURE_NAME

```

```

    def __repr__(self):

```

```

        # f-строки быстрее, но по заданию нужно использовать format
        return "{} {} цвета шириной {} и высотой {}. Площадь: {:.2f}".format( #
pylint: disable=consider-using-f-string
            self.name, self.color.color, self.width, self.height, self.area()
        )

```

```

ngle.py`

```

```

`lab_python_oop/square.py`

```

```

"""

```

```

Модуль с классом Квадрат
"""

```

```

from typing import Union
from .rectangle import Rectangle

class Square(Rectangle):
    """
    Класс Квадрат (наследуется от Прямоугольника)
    """

    FIGURE_NAME: str = "Квадрат"

    def __init__(self, side: Union[int, float], color: str):
        # Вызываем конструктор родительского класса
        super().__init__(side, side, color)

    @property
    def name(self) -> str:
        """Название фигуры"""
        return self.FIGURE_NAME

    def __repr__(self):
        # f-строки быстрее, но по заданию нужно использовать format
        return "{} {} цвета со стороной {}. Площадь: {:.2f}".format( # pylint:
disable=consider-using-f-string
            self.name, self.color, self.width, self.area()
        )

```

Скриншот работы приложения

```
(.venv) cunev@ROGStrixG814JIR:~/VSCode/ParadigmsAndConstructsOfProgrammingLanguages/lab2$ python ./main.py
=== Лабораторная работа №2 ===
=== Объектно-ориентированные возможности Python ===

Информация о геометрических фигурах:
1. Прямоугольник синий цвета шириной 13 и высотой 13. Площадь: 169.00
2. Круг зеленый цвета радиусом 13. Площадь: 530.93
3. Квадрат красный цвета со стороной 13. Площадь: 169.00

=== Демонстрация внешнего пакета colorama ===
Этот текст красного цвета
Этот текст синего цвета
Этот текст зеленого цвета
Этот текст ярко-белый

=== Дополнительная информация ===
Тип прямоугольника: Rectangle
Тип круга: Circle
Тип квадрата: Square

=== Проверка наследования ===
Квадрат является прямоугольником: True
Площадь квадрата: 169.00
```

Рисунок 1 Результат работы main.py

```
(.venv) cunev@ROGStrixG814JIR:~/VSCode/ParadigmsAndConstructsOfProgrammingLanguages/lab2$ pytest
===== test session starts =====
platform linux -- Python 3.12.3, pytest-8.4.1, pluggy-1.6.0
rootdir: /home/cunev/VSCode/ParadigmsAndConstructsOfProgrammingLanguages/lab2
collected 6 items

test_figures.py ..... [100%]

===== 6 passed in 0.01s =====
```

Рисунок 2 Тесты геометрических фигур